

SpecDVFS: An Empirical Study of Per-Phase Dynamic Voltage and Frequency Scaling for Energy-Efficient LLM Speculative Decoding

A project report : motivation, experiments, results and conclusions

Sifat Singh Oberoi
sifat24556@iiitd.ac.in

Shreyas Modi
shreyas24544@iiitd.ac.in

Mannat Raj Singh
mannat24333@iiitd.ac.in

Abstract

Speculative decoding (SD) accelerates large-language-model (LLM) inference by having a small *draft* model propose tokens that a large *target* model then *verifies* in a single batched pass. A widely repeated intuition holds that the draft phase is memory-bound (so its clock can be lowered “for free”) while the verify phase is compute-bound (so it must run at full clock), which suggests that per-phase Dynamic Voltage and Frequency Scaling (DVFS) should save GPU energy at negligible latency cost. This intuition was backed by a similar phase-level asymmetry between the *prefill* and *decode* stages of ordinary, non-speculative autoregressive decoding which has already been shown to work in practice. Multiple independent studies report that decode-phase or fully phase-aware GPU frequency scaling on vanilla decoding yields substantial energy savings (on the order of 30–45%) at only a small latency cost, because the decode stage is memory-bound and largely insensitive to GPU frequency while prefill is compute-bound and clock-sensitive [8–10]. **SpecDVFS** set out to build the analogous per-phase mechanism for the draft/verify split of speculative decoding specifically, in a production inference engine (vLLM). We did not ourselves re-evaluate the vanilla prefill/decode case, which this prior work has already established, and instead set out to measure whether the predicted energy savings actually materialise on real hardware with minimal latency overhead for the draft/verify phase split.

We report a thorough, largely *negative* result. We first designed and validated an entire per-phase DVFS control system without a GPU (a decoupled decision-logic layer plus a hardware clock-lock layer), then ran it across 95+ measured configurations on a rented RTX 3090. Three independent probes: a broad 75-run pilot (two draft pairs $\times$ three draft lengths $\times$ three datasets $\times$ four policies), a controlled 20-run streaming-multiprocessor (SM) restriction experiment and a 40-run batch-size sweep - converge on the same conclusion. On unmodified hardware the premise is *false*: both phases measure below the roofline ridge point (draft ≈ 41.1 , verify ≈ 61.3 , ridge ≈ 75.9 FLOPs/byte), i.e. both are memory-bound. Per-phase DVFS does deliver real and reproducible GPU-energy savings (3–10%), but these are consistently outweighed by a large latency penalty (46–88%), so the energy–delay product (EDP) favours the no-DVFS baseline at every operating point we tested. When we used SM restriction as a controlled instrument to *force* the compute/memory split the theory requires, the predicted benefit still failed to appear-savings declined monotonically rather than peaking inside the “premise-true” window. A draft-clock sweep shows

why: the draft phase is *clock-limited in practice* despite its memory-bound roofline classification, so the effect is an ordinary power-versus-latency trade-off rather than the compute/memory asymmetry to exploit which we had assumed. We also find that a sophisticated entropy-based adaptive policy never separates from a trivial static one, and we document a methodological capture bug and an unresolved measurement-tool disagreement in full. We present the project chronologically, including the failed and abandoned approaches, so that the engineering and research process is as visible as the final numbers.

1 Introduction

Autoregressive decoding for large language models is memory- and bandwidth dominated and energy hungry: each new token requires streaming the entire weight set of a multi-billion-parameter model out of GPU DRAM. *Speculative decoding* (SD) [1, 2] attacks the latency side of this problem by splitting generation into two phases. A small, cheap *draft* model proposes a block of γ candidate tokens autoregressively; a large *target* model then *verifies* all γ candidates in one batched forward pass and accepts the longest correct prefix. When the draft agrees with the target often enough, several tokens are emitted per expensive target pass, thus, leading to fall of end-to-end latency.

This project, **SpecDVFS**, was motivated by a separate observation: the two SD phases appear to have very different hardware characteristics. The draft phase runs a tiny model one token at a time and looks memory-bound; the verify phase scores many candidates at once and looks compute-bound. If that is true, then *per-phase* Dynamic Voltage and Frequency Scaling (DVFS)-lowering the GPU core clock during drafting and restoring it during verification-should shave energy off the draft phase almost for free, because a memory-bound kernel is waiting on DRAM rather than on the core clock. The verify phase, being compute-bound, would keep its full clock and lose no speed. The net effect would be a reduction in GPU energy at negligible latency cost.

The purpose of this report is to document, in full and in chronological order, what happened when we tried to realise and validate that idea. The short version is that *the premise does not hold on real hardware*, and that even when we engineered the hardware so that it *did* hold, the predicted benefit did not appear. The longer version-the system we built, the three experimental campaigns we ran, the diagnostics that explained the results, the bugs we found, and the measurement issues that remain open-is the substance of the report.

1.1 Contributions

This report makes the following contributions:

- (1) **A complete, GPU-free-testable per-phase DVFS control system for vLLM speculative decoding.** We decouple a pure decision-logic layer (which clock should each phase use) from a thin hardware layer (which issues the NVML clock-lock call), so that every control policy is unit-testable on a laptop with no GPU. The system implements six DVFS policies, a leading-indicator entropy signal with automatic per-model-pair calibration, and a monkey-patch that wires all of this into vLLM 0.6.6’s speculative-decoding worker without editing vLLM source (Section 5).
- (2) **A broad empirical characterisation of per-phase DVFS** across two draft/target pairs, three draft lengths ($\gamma \in \{5, 12, 18\}$), three datasets (GSM8K, HumanEval, CNN/DailyMail), and four policies-establishing that savings exist ($\sim 6\text{--}9\%$ mean GPU energy) and quantifying their latency cost ($\sim 74\%$ mean overhead) (Section 7).
- (3) **A roofline premise check and a controlled SM-restriction instrument.** We show the premise fails on a stock RTX 3090, then use NVIDIA MPS to lower the roofline ridge point and *force* the verify phase compute-bound. The central negative result is that energy savings do *not* respond to this premise—they decline monotonically as the premise is switched on and then broken the other way (Sections 8–9).
- (4) **A realistic batch-size sweep, a methodological bug we found while cross-checking it, and an honest account of an unresolved measurement disagreement.** We show the EDP verdict is robust across batch sizes; we trace and fix a capture bug that silently pinned all roofline runs to a single prompt; and we document a ~ 10 percentage-point CodeCarbon-versus-NVML disagreement on the *sign* of net savings, together with the argument for why our headline conclusion survives it (Sections 10–11).

1.2 How to read this report

The report is organised chronologically wherever possible, so that it tells the story of how the project evolved. For each major component or experiment we follow a consistent five-part pattern: (i) how the idea arose, (ii) why we expected it to work, (iii) how we implemented it, (iv) whether it worked, and (v) if it did not, why not (with concrete evidence where we have it, and an explicit statement of uncertainty where we do not). Section 2 gives the necessary background. Section 3 states the central hypothesis. Sections 4–11 walk through the project in phases. Sections 12–14 discuss what the evidence points to, threats to validity, and conclusions.

2 Background and Related Work

2.1 Speculative decoding

Speculative decoding [1, 2] preserves the target model’s outputs under greedy decoding and its output distribution under corrected sampling. The key quantity is the *acceptance rate*, α : the fraction of proposed draft tokens that the target accepts. Higher α means

more tokens emitted per target pass and thus more speedup. α depends primarily on how closely the draft model matches the target model: a better-aligned draft yields a higher acceptance rate. It also depends on the draft length, γ . As γ increases, the draft proposes tokens further into the future, where the draft and target are more likely to disagree. Consequently, the average acceptance rate across the proposed tokens typically decreases with increasing γ . Both dependencies are central to our study: we deliberately vary the draft length γ and the draft model to produce different acceptance rates α (Section 6).

We use vLLM’s [3] implementation of standard speculative decoding with a *separate* draft model (as opposed to self-speculative variants; see Section 2.4). In vLLM 0.6.6, the acceptance rate is defined as $\alpha = \text{num_accepted_tokens} / \text{num_draft_tokens}$, where the numerator and denominator are taken directly from the speculative sampler’s internal counters. We adopt this definition throughout so that our per-iteration and per-run measurements are directly comparable with vLLM’s reported acceptance rate (Section 5.6).

2.2 The speculative-decoding energy anomaly

One of our starting motivation was an empirical anomaly reported by Dutta et al. [5] (EACL 2026): speculative decoding, while it can reduce latency, does not straightforwardly reduce GPU energy consumption. It can even increase it under some configurations. This anomaly is what makes an energy-targeted intervention interesting in the first place: if SD already minimised energy, there would be nothing for DVFS to recover. We therefore treat this anomaly as established prior work and cite it.

2.3 DVFS and the roofline model

Dynamic Voltage and Frequency Scaling trades compute throughput for power: lowering the core clock (and voltage) reduces dynamic power roughly super-linearly, but also slows any computation that is bound by the core clock. Whether a given kernel is *clock-bound* is captured, to first order, by the *roofline model* [4]. Define a kernel’s *arithmetic intensity* as

$$I = \frac{\text{FLOPs performed}}{\text{bytes of DRAM traffic}} \quad [\text{FLOPs/byte}], \quad (1)$$

and the machine’s *ridge point* as

$$I^* = \frac{\text{peak FLOP/s}}{\text{peak memory bandwidth}} \quad [\text{FLOPs/byte}]. \quad (2)$$

A kernel with $I < I^*$ is *memory-bound* (limited by DRAM bandwidth, so raising the core clock buys little), and one with $I \geq I^*$ is *compute-bound* (limited by the math units, so the core clock helps). The premise behind per-phase DVFS is precisely that $I_{\text{draft}} < I^* \leq I_{\text{verify}}$: draft on the memory-bound side (downclock for free), verify on the compute-bound side (needs full clock). Crucially, I^* is a property of the *machine*, not of the kernel – a fact we later exploit to move the ridge without changing the workload (Section 9).

2.4 Related systems and where we differ

Two recent works are close enough to require explicit differentiation.

PELM [6] (SenSys 2026). PELM also combines speculative decoding with DVFS, using reinforcement learning, but targets a different regime. PELM is built for low-power edge platforms (e.g. NVIDIA Jetson AGX Orin/Orin Nano, 7–60 W) where *thermal* constraints are central, and it uses *self*-speculative decoding (Layer-Skip), which requires specially pre-trained models and exploits the observation that not every token needs full model depth. SpecDVFS instead targets high-performance discrete/data-center GPUs (300 W+) where energy efficiency matters more than thermals, and uses *standard* SD with a separate draft model, making it compatible with any model pair without special pre-training. We try to exploit the asymmetric compute character of the draft and verify phases instead of the per-token model depth.

GELATO [7] (Tang et al., 2026). GELATO studied the link between the draft model’s output *entropy* and the acceptance rate α , and found it dependable: confidence and acceptance track along a predictable curve, well approximated by an exponential decay $\alpha \approx a \cdot e^{bH}$ where H is the draft’s Shannon entropy. GELATO reports $a = 1.0$, $b = -0.35$ for a Qwen2.5 pair. We adopt this functional form as the basis of our leading-indicator `adaptive_entropy` policy and re-fit (a, b) per model pair (Section 5.5).

3 The Central Hypothesis and Its Rationale

How the idea arose. The project grew out of the SD energy anomaly (Section 2.2): if SD wastes energy, can a phase-aware clock policy recover it? The specific per-phase hypothesis is the standard justification for *any* phase-aware DVFS scheme and it is attractive because it promises substantial energy savings for minimal latency costs.

The hypothesis (what we believed going in).

- *Draft phase* $\rightarrow$ memory-bound $\rightarrow$ downclock it to lose little time and save power.
- *Verify phase* $\rightarrow$ compute-bound $\rightarrow$ keep it at full clock as it needs the FLOPs.
- *Net effect* $\rightarrow$ energy saved with minimal latency cost, as we never slows down the phase that actually needs speed.

Why we expected it to work. The intuition rests on the roofline argument of Section 2.3. The draft model is small and runs one token at a time; its forward passes stream weights from DRAM with little arithmetic reuse, which *looks* memory-bound. The verify pass scores $\gamma + 1$ candidate positions for every sequence in a single batched matrix multiply, which amortises each weight read over more FLOPs and *looks* compute-bound. If both intuitions are correct, the phases sit on opposite sides of the ridge point, and the free lunch follows directly from the model.

The same compute/memory split is the widely accepted characterization of *vanilla* (non-speculative) autoregressive decoding: the *prefill* phase processes the entire input prompt in one batched forward pass (compute-bound, high arithmetic intensity, FLOPs scale with prompt length while weights are read once) and the token-by-token *decode* phase re-reads the full weight set from DRAM for every single new token (memory-bound, low arithmetic intensity). This prefill/decode asymmetry is well established and is exactly the kind of phase-aware DVFS opportunity we were trying to generalise: on vanilla decoding, treating prefill and decode as distinct roofline regimes and clocking each accordingly is known to work.

Draft and verify are structurally analogous to decode and prefill, one token at a time versus one batched multi-position pass, so the compute/memory classification applies from vanilla decoding to speculative decoding.

The load-bearing assumption The entire benefit hinges on the ordering $I_{\text{draft}} < I^* \leq I_{\text{verify}}$. If that ordering fails, if verify is also memory-bound-then holding verify at f_{high} is simply spending power for no speedup and any energy “savings” from downclocking draft must be scrutinised, because they might be an ordinary power/latency trade rather than the predicted hypothesis. This is exactly why we do a roofline premise check *before* along with the pilot instead of committing to a full sweep.

4 Phase I - Initial Replication Attempt and Strategic Pivot

The first phase of the project (early June 2026) attempted to reproduce the SD energy anomaly on locally available hardware and quickly ran into a hardware wall.

4.1 How the idea arose and why we expected it to work

The initial plan was conservative: before proposing DVFS as a solution, replicate the energy-versus-speed findings of Dutta et al. [5] on our own setup, so that our later DVFS numbers would have a baseline. We planned to run LLaMA-3.1-8B (target) and LLaMA-3.2-1B (draft) under different GPU clock speeds on a gaming laptop with an RTX 4070, mapping how SD affects energy.

4.2 How we implemented it

We wrote a custom Python benchmarking script comparing vanilla decoding against speculative decoding and running on the GSM8K reasoning dataset. Hardware behaviour was monitored with `htop` under WSL and Windows Task Manager, purely to confirm the intended resources were being exercised.

4.3 Why the Per-Phase DVFS Hypothesis Did Not Hold

Our evaluation indicates that the standard per-phase DVFS intuition did not hold under this hardware for two distinct reasons:

(1) *DVFS was not controllable on the laptop.* We found we could not programmatically control GPU clocks on the laptop chip at all: NVIDIA locks clock offsets, voltage scaling, and power limits on laptop GPUs at the BIOS and driver level (to prevent overheating). Since the whole project depends on setting clocks via `nvidiaDevice-SetGpuLockedClocks`.

(2) *SD was dramatically slower than vanilla on the laptop.* The benchmark itself produced an inverted result: vanilla decoding took ≈ 40 minutes over the dataset (as expected), while speculative decoding (denoted COGA-5, $\gamma = 5$) took ≈ 4 hours, and a longer-draft variant (COGA-10) was so slow we aborted it after 3 hours. Speculative decoding ran much slower than vanilla—the opposite of a speedup. We diagnosed several compounding laptop-specific causes:

Table 1: Hardware specification required for SpecDVFS, derived from the Phase I roadblocks. The binding requirements are programmatic clock control and enough VRAM to hold both models in bf16 without fragmentation.

Requirement	Value
GPU	A100 / H100 / A6000 / A5000 / RTX 5090 / RTX 4090 / RTX 3090 desktop (not laptop, not MIG-slice, not T4)
VRAM	≥ 24 GB
System RAM	≥ 32 GB
Storage	≥ 200 GB free
OS	Native Linux
Internet	access to huggingface.co for model downloads
Access	sudo or persistence-mode pre-enabled; exclusive GPU;
Clock control	permission to call <code>nvmlDeviceSetGpuLockedClocks</code> ; on HPC clusters this needs admin confirmation

- **8 GB VRAM limit.** Holding two models caused major memory fragmentation; one CPU core saturated at 100% simply managing memory allocation, starving the GPU of work.
- **Miss penalty amplified by bandwidth.** On a server-class GPU with ~ 768 GB/s bandwidth the cost of a rejected draft block is near-zero, turning even a modest acceptance rate into a $\sim 1.4\times$ speedup; the laptop’s PCIe $\times 8$ link plus power throttling makes every miss far more expensive in time.
- **Consistency with the literature.** Even on a research server, very large targets (e.g. LLaMA-70B) can show $< 1\times$ “speedup” from SD, which directly matches the overhead warnings in Dutta et al. [5].

4.4 Redirecting the project

The Phase I failures pointed towards two decisions:

Drop the replication. We chose to treat the anomaly itself as settled ground and build forward from it, with these three reasons: (i) the SD energy-waste anomaly is already established and peer-reviewed (EACL 2026), so it suffices to cite it; (ii) direct number-for-number comparison is methodologically flawed because their HuggingFace / A5000 setup differs from our intended vLLM / server-GPU setup; and (iii) our *true* baseline would organically be “SD without DVFS” *within our own experiment matrix*, which is a cleaner comparison than a cross-setup replication.

We understood that we need to escalate the hardware. The laptop’s locked DVFS and 8 GB VRAM were fundamental roadblocks. We therefore specified the hardware the project actually needed (Section 5.1) and began the search for a machine that met it.

5 Phase II - Infrastructure: Hardware Acquisition and a GPU-Free System Design

With the laptop ruled out, the project entered a new phase (mid-June 2026) with two parallel tracks: securing a machine that satisfied Table 1, and pre-validating the entire experimental harness without a GPU. Because GPU time was metered and expensive, every component was debugged in advance so that the rented GPU could be used exclusively for data collection.

Table 2: Hardware options explored, in order, and why each did or did not meet the requirements. Vast.ai was the first to satisfy all binding constraints.

Option	Outcome
Laptop RTX 4070	Rejected: locked DVFS (BIOS/driver); 8 GB VRAM $\rightarrow$ fragmentation (Section 4).
Desktop GTX 1650 (4 GB)	Rejected: insufficient VRAM (need 24 GB+), no DVFS control, weak PCIe bandwidth.
RunPod AI	Rejected: contacted for root/sudo access for programmatic DVFS (clock offsets, power limits, voltage); did not satisfy the access requirement.
Lambda Labs	Rejected: emailed to confirm low-level hardware control (unlocked clock offsets via <code>nvidia-smi</code>); did not satisfy the root-access requirement.
Vast.ai	Accepted: contacted the Vast.ai team regarding root/sudo access for programmatic DVFS control; they could not confirm availability and advised us to verify on rented instances. We purchased \$5 of credit, tested multiple instances, and confirmed that selected machines provide root/sudo, enabling DVFS via <code>nvidia-smi</code> while also satisfying the 24 GB+ VRAM requirement.

5.1 The hardware acquisition search

How the idea arose / why. Table 1 is strict, and each candidate had to be checked against the binding requirements—programmatic clock control and ≥ 24 GB VRAM—rather than headline specs. We evaluated several options in sequence (Table 2).

Outcome. We proceeded on Vast.ai with a rented RTX 3090 (24 GB, ≈ 23.69 GiB usable). The RTX 3090 turned out to be sufficient for the pilot. VRAM was not the limiting resource for this study, so a larger 40–48 GB card remained optional (it would only buy longer-context CNN/DailyMail, higher throughput, and CUDA graphs). All confirmed GPU lock anchors and peak numbers in this report ($f_{\text{high}} = 1935$ MHz, $f_{\text{low}} = 735$ MHz; peak ≈ 71 TFLOP/s FP16 / 936 GB/s) are for this RTX 3090.

5.2 Design principle: decouple decision logic from hardware

Motivation. Because GPU time was metered and expensive, we wanted to eliminate as much on-GPU debugging as possible. The key insight was that a DVFS controller consists of two separable concerns: a *policy* (“given the current phase and acceptance rate, which clock should be selected?”) and an *actuator* (“issue the NVML call that pins the clock”). The policy is hardware-independent and can therefore be developed and validated without a GPU; only the actuator requires direct interaction with the hardware.

Implementation Details. We split the system into two layers:

- (1) A *decision-logic layer* (`controller/core.py`) that decides which clock each phase should run at and logs every decision, *without ever calling NVML*. This layer is fully unit-testable on any machine, GPU or not.
- (2) A *hardware layer* (`vllm_hooks/dvfs_controller.py`) that subclasses the logic layer and overrides exactly one method, `set_frequency_mhz()`, with a real `pynvml.nvmlDeviceSetGpuLockedClocks()` call plus clock-snapping and a reset helper.

Every logical branch therefore lives in the GPU-free layer and is covered by CPU unit tests; the hardware subclass adds the physical effect and nothing else. This is what let us build and validate the whole codebase before ever touching the GPU.

5.3 The DVFS controller and its six policies

Implementation. The decision-logic layer is built from three small classes that each do one job, and hand off to each other in sequence.

First, an AcceptanceRateTracker watches how often the draft model’s guesses get accepted and keeps a running, smoothed estimate of α rather than reacting to every single iteration. It does this with an exponential moving average, $\alpha_{\text{ema}} \leftarrow d \cdot \alpha_{\text{ema}} + (1-d) \cdot \alpha_{\text{obs}}$, using a decay of $d = 0.7$. In practice this means recent iterations carry most of the weight, but one unusually good or bad iteration can’t swing the estimate on its own.

Second, a FrequencyMapper takes that acceptance-rate estimate and decides what clock speed the *verify* phase should run at. The draft phase doesn’t need this logic at all - it always gets f_{low} , whether it’s drafting normally, rolling back, or falling back to non-speculative decoding. The mapping for *verify* is deliberately simple: a straight line between a floor and a ceiling clock (Eq. 3), with the floor set at $f_{\text{floor}} = \lfloor 0.6 f_{\text{high}} \rfloor$.

Third, a SimulatedDVFSController ties the two together. It is what the monkey-patch actually talks to: it exposes the hook points the patch calls into, picks which policy to run based on a mode string, and keeps a log of every decision it makes - which phase, what frequency, what α triggered it - so we have a full trace of the controller’s behaviour after the fact.

Putting the mapper’s logic into an equation, the *verify*-phase clock is

$$\text{verify_freq}(\alpha) = \begin{cases} f_{\text{high}}, & \alpha \geq \alpha_{\text{high}}, \\ f_{\text{floor}}, & \alpha \leq \alpha_{\text{low}}, \\ f_{\text{floor}} + t(f_{\text{high}} - f_{\text{floor}}), & \text{otherwise,} \end{cases} \quad (3)$$

where $t = (\alpha - \alpha_{\text{low}}) / (\alpha_{\text{high}} - \alpha_{\text{low}})$ and the thresholds are $\alpha_{\text{low}} = 0.3$ and $\alpha_{\text{high}} = 0.7$. In plain terms: when the draft model is agreeing with the target often enough ($\alpha \geq 0.7$), *verify* gets the full clock; when agreement is poor ($\alpha \leq 0.3$), it drops to the floor clock; anywhere in between, it slides linearly along that line. Whatever value comes out gets snapped to the nearest clock level the hardware actually supports - the RTX 3090 exposes 127 discrete graphics-clock steps, so the mapper’s continuous output always lands on a real, settable value. Listing 1 shows the core of this mapping in code.

Listing 1: The *verify*-phase clock mapping (*core.py*). Draft and fallback always return f_{low} ; only *verify* is α -dependent.

```
def verify_freq(self, alpha):
    if alpha >= self.alpha_high: # 0.7
        return self.f_high # 1935 MHz
    if alpha <= self.alpha_low: # 0.3
        return self.f_floor # round(0.6 * f_high)
    t = (alpha - self.alpha_low) / (self.alpha_high - self.alpha_low)
    return int(round(self.f_floor + t * (self.f_high - self.f_floor)))

def draft_freq(self): # draft/rollback/fallback
    return self.f_low # 735 MHz, always
```

The six DVFS policies. We implemented six DVFS modes to separate two questions: whether phase-aware DVFS provides any benefit at all, and, if it does, how sophisticated the policy needs to be. Table 3 summarises the six modes. A single mode-aware `on_verify_start()` method selects the active policy based on the configured mode string, allowing the monkey-patch to remain identical across all experimental conditions while varying only the frequency-selection logic.

The entropy signal. The `adaptive_entropy` policy predicts α from the draft model’s confidence *before* verification runs, using GELATO’s exponential form:

$$\hat{\alpha} = \max(0, \min(1, a e^{bH})), \quad (4)$$

where H is the mean Shannon entropy of the draft’s proposed-token distributions. The distinction from `adaptive_alpha` is timing: α from the sampler is a *lagging* signal (only known *after* verification of the previous iteration finishes), whereas H is available *between* draft and *verify* within the same iteration, so the *verify* clock can be set proactively rather than one iteration late.

5.4 Wiring DVFS into vLLM: the monkey-patch

Motivation. We needed the GPU clock to change exactly at the draft→*verify* boundary so that each phase would execute at its intended frequency, without modifying vLLM’s source code. Our solution was a monkey-patch that wraps the specific methods in vLLM’s speculative-decoding worker marking each phase boundary.

Implementation. Confirmed against the installed vLLM 0.6.6 `spec_decode` source, the patch wraps four call sites and installs one forward hook (Table 4). Phases are detected by *wrapping the calls*, not by polling the GPU: the clock is set immediately before the wrapped method launches its kernels, so they launch at the intended frequency. Rollback is deliberately *not* wrapped-in vLLM the bonus/corrective token is produced inside the scorer’s single forward pass, so draft and *verify* are the only distinct GPU phase boundaries.

Fork-safety. NVML handles are not fork-safe, and vLLM spawns its GPU worker as a separate process. The patch therefore exposes an `install(controller_factory)` entry point that patches `SpecDecodeWorker.init_device` at the class level; the wrapper runs *inside* each worker process, after the original `init_device` has built the scorer/proposer/sampler, and only then constructs the controller (so `nvmlInit()` runs in the right process). Critically, `apply_patch` is applied for *every* condition, including the disabled baseline, so all conditions take an identical code path with only frequency-setting toggled-the baseline carries the same wrapping overhead as the treatment. Listing 2 shows the draft/*verify* wrappers.

Listing 2: The draft/*verify* wrappers (*patch_spec_decode.py*, simplified). The clock is set immediately before the original method launches its kernels.

```
def draft_wrapper(*args, **kwargs):
    controller.on_draft_start() # set f_low
    return orig_draft(*args, **kwargs)

def verify_wrapper(*args, **kwargs):
```

Table 3: The six DVFS policies (modes). *off* is the SD-without-DVFS baseline. The two adaptive policies differ in whether they react to the *previous* iteration’s measured α (lagging) or predict *this* iteration’s α from draft entropy (leading). *fixed_low* and *coarse* are ablations.

Mode	Signal	Behaviour
off	-	No DVFS. Controller disabled; hooks no-op; GPU stays at its default governor. The baseline every DVFS condition is measured against.
two_level	none (static)	Phase-aware but static: f_{low} on draft, f_{high} on verify, α ignored. The reference every adaptive policy is judged against.
adaptive_alpha	lagging α	Draft at f_{low} ; verify clock = verify_freq(EMA of last iteration’s α). Reactive.
adaptive_entropy	leading H	Draft at f_{low} ; verify clock = verify_freq($\hat{\alpha}$) with $\hat{\alpha} = a e^{bH}$ computed from the draft’s output entropy H <i>within the same iteration</i> . Proactive; needs the per-pair (a, b) fit. Degrades to adaptive_alpha if the entropy hook never fires.
fixed_low	none	Naive global low: f_{low} on <i>both</i> phases (verify too). The most aggressive energy-saving attempt; used to confirm verify’s clock sensitivity.
coarse	lagging α	Granularity ablation: one clock chosen per ~ 100 ms window and held across <i>both</i> phases (no per-phase drop), to test whether reacting at per-phase granularity even matters.
collect	-	Not a deployment policy: a calibration pre-pass. Runs DVFS-off so the GPU stays at default clock while the patch records (H, α) pairs for the entropy fit (Section 5.5).

Table 4: The monkey-patch hook points (all in `vllm/spec_decode/`). Wrapping these lets the controller set the clock at each phase boundary and read α where acceptance is actually decided.

Wrapped method	Role
<code>proposer_worker.get_spec_proposals</code>	DRAFT phase $\rightarrow$ set f_{low} before it.
<code>scorer.score_proposals</code>	VERIFY phase $\rightarrow$ set verify clock before it.
<code>SpecDecodeWorker._verify_tokens</code>	Rejection sampling: where acceptance is decided $\rightarrow$ read α here (a counter delta), not from <code>score_proposals</code> (which returns only logits).
<code>SpecDecodeWorker._run_no_spec</code>	Non-speculative fallback decode $\rightarrow$ set f_{low} .
<code>draft_lm_head</code> forward hook	Compute mean draft entropy $H \rightarrow$ <code>controller.last_entropy</code> (leading signal + calibration source).

```

controller.on_verify_start(alpha=controller.tracker.estimate) #
mode-aware
return orig_verify(*args, **kwargs)

def verify_tokens_wrapper(*args, **kwargs):
    before = metrics_reader.snapshot_counts(sampler)
    out = orig_verify_tokens(*args, **kwargs) # acceptance decided
    here
    alpha = metrics_reader.iteration_acceptance_rate(
        before, metrics_reader.snapshot_counts(sampler))
    if alpha is not None:
        controller.record_acceptance_rate(alpha) # feeds next iter's
        clock
    return out

```

5.5 Entropy–acceptance calibration

Motivation. The adaptive_entropy policy needs the coefficients (a, b) of Eq. 4 for the specific model pair in use; GELATO’s $(1.0, -0.35)$ was fit for a different pair. Rather than hard-code them, we fit them empirically, once per pair.

Implementation. A calibration pre-pass runs in collect mode (DVFS off, so the clock does not perturb the measurement). A forward hook on the draft `lm_head` records the per-token Shannon entropy $H = -\sum_v p_v \log(p_v + 10^{-10})$; at each iteration boundary,

the buffered token entropies are averaged and paired with that iteration’s measured α . We then fit $\alpha = a e^{bH}$ with SciPy’s `curve_fit` (bounds $a \in [0, 5]$, $b \in [-10, 0]$), reporting R^2 and standard errors, and cache the result to `calibration/fitted_<pair>.json`; if a pair yields no usable pairs or a poor fit, we fall back to the GELATO baseline $(1.0, -0.35)$. Two design choices are worth noting:

- **Calibrate across all dataset types, not one.** A single dataset risks narrow entropy coverage (GSM8K’s formulaic steps, for instance), which would leave the curve poorly constrained outside that range. We spread a total budget of 128 prompts across GSM8K, HumanEval, and CNN/Daily-Mail (roughly 43 each), at $\gamma = 5$, DVFS off. Because each prompt yields many (H, α) pairs (one per decode step), the fit still sees far more than its minimum sample count.
- **The live and calibrated entropy are the same quantity.** The live adaptive_entropy hook computes H with exactly the same definition as the calibration collector, so the fitted coefficients describe the quantity the controller actually consumes.

The fit runs automatically the first time a pair that needs it appears in an experiment.

5.6 Energy measurement, determinism and the evaluation pipeline

Two independent energy measurements. Every run records GPU energy two ways, and the two methods differ in a way that matters for how we read disagreements between them. *NVML* is a direct hardware measurement: the driver samples the GPU’s on-board power sensor at 100 Hz (a 10 ms trace) and we integrate that trace over wall time. Its main failure mode is *undersampling*—a 10 ms sample period can miss power spikes shorter than that window, which would bias the reported energy *downward* during bursty phases. *CodeCarbon*, by contrast, is a *software estimator*: it reads GPU power from NVML as well, but it *does not* measure CPU and RAM power directly—it infers them from the package’s rated TDP (thermal design power) and OS-reported utilization, then adds a fixed baseline/idle draw for the measurement interval. This is where the two methods can diverge in ways that are not just noise.

TDP is a *worst-case* rating, not a typical operating draw, so scaling it by utilization can overstate actual CPU/RAM power whenever the true power-versus-utilization curve is sub-linear (common on modern chips with aggressive power gating). The bias is largest for *short, bursty* runs—exactly what per-iteration GPU decoding looks like—because a brief burst of high utilization can trigger a TDP-scaled estimate for the whole sampling interval, while the fixed idle/baseline term is charged regardless of how much of that interval was actually idle. In short: NVML risks *missing* real power draw; CodeCarbon risks *inventing* power draw that was never actually pulled from the wall. We keep both numbers rather than picking one, and we flag where they disagree rather than silently trusting either. This dual measurement later surfaces a genuine, unresolved disagreement (Section 11.2).

Acceptance rate as the single source of truth. α is defined once, in `metrics_reader`, as vLLM’s own `num_accepted/num_draft` read off the speculative sampler’s counters. The same definition serves the per-iteration value (a counter delta the controller consumes) and the per-run value (the cumulative ratio in the result JSON). We explicitly avoid parsing the verifier’s output tensor, which reflects the *emitted* prefix and would disagree with vLLM’s metric in mixed batches; a tensor-based estimate is retained only as a labelled diagnostic. Listing 3 shows the per-iteration computation, which returns `None` (never 0.0) on a non-speculative step so that a meaningless value never poisons the EMA or the calibration fit.

Listing 3: Per-iteration α as a counter delta (`metrics_reader.py`). `None` means “no usable measurement”; a real all-rejected iteration is a genuine 0.0.

```
def iteration_acceptance_rate(before, after):
    if not before or not after:
        return None
    d_draft = after["draft"] - before["draft"]
    if d_draft <= 0: # fallback / non-spec step
        return None
    d_acc = after["accepted"] - before["accepted"]
    return clamp01(d_acc / d_draft) # in [0, 1]
```

Determinism is load-bearing. All runs use temperature 0 and a fixed seed (42). This makes the accept/reject sequence *bit-identical* across DVFS conditions, because DVFS changes clock *timing*, not *computation*. Determinism is what makes the energy comparison valid: any energy difference between off and a DVFS condition is attributable to the clock policy, not to run-to-run variation in *what* was generated. We verify this directly—total token counts match across conditions and α varies by under 1% across the SM-sweep runs (Section 9). It also has a consequence we did not anticipate and had to design around (Section 6.1): at fixed γ , α is *constant* across conditions.

Run hygiene. Each run does 5 warm-up prompts, enforces a temperature ceiling of 75 °C with a 30 s cool-down between runs, and records the 10 ms monitor trace, per-run energy (both meters), wall time, token counts, and α into one JSON per run.

Evaluation pipeline (local only). The GPU produces only *raw* data (per-run JSONs and profiling CSVs); all analysis runs locally, in three stages: `compute_metrics.py` turns each JSON into one tidy CSV row with derived metrics (energy in Wh, tokens/s, Wh-per-1k-tokens, and the energy-delay product $EDP = \text{energy} \times \text{wall time}$); `aggregate_results.py` computes the headline savings

of each DVFS condition versus off within each cell (energy saving %, latency overhead %, EDP ratio, and a Welch *t*-test *p*-value); and `generate_figures.py` renders the plots. Keeping analysis off the GPU saves metered time and keeps the analysis reproducible.

5.7 Pre-GPU validation

Before any paid GPU time, the whole system was validated on a laptop: a unit-test set covering all controller modes and the patch’s α path (no GPU, vLLM, or datasets), and a fully mocked end-to-end pilot that exercises the run matrix, JSON schema, and resume logic with fakes. Phase 0 (laptop, free) → Phase 1 (GPU provisioning + a clock-lock GO/NO-GO check) → Phase 2 (pilot smoke test) → Phase 3 (profiling: per-phase timing and roofline capture) → Phase 4 (pull raw data home, analyse locally). The full sweep is gated on both the pilot toggling the clock correctly and the roofline verdict being GO. Several engineering decisions and abandoned approaches from this phase are recorded in Table 5.

Table 5: Key engineering decisions and abandoned approaches from Phase II (GPU-free build) and the first GPU sessions. Documented because several shaped the measured configuration and rule out otherwise-tempting alternatives.

Decision	Rationale / what was abandoned and why
enforce_eager=True	CUDA-graph capture added ~1.27 GB <i>after</i> KV-cache sizing → OOM; eager reclaims it and removes graph-replay timing artifacts (methodologically cleaner). <i>Abandoned: CUDA graphs.</i>
max_num_seqs=8	Directly bounds the rejection-sampler tensor (scales batch $\times \gamma \times \text{vocab}$) that OOM'd in <code>_get_recovered_probs</code> .
max_model_len=2048	Halves KV-cache per sequence versus 4096 (which contributed to OOM); still exceeds GSM8K/HumanEval prompt +256 generation. <i>Abandoned: 4096.</i>
vLLM pinned 0.6.6	Required for the patch's <code>spec_decode</code> internals (8 name constants). A newer vLLM would fix bitsandbytes but break the patch and cannot load the pinned models the same way.
transformers pinned < 5.0.0	v5's tokenizer backend dropped <code>all_special_tokens_extended</code> , which vLLM 0.6.6 calls.
Code delivery via tar ssh	A bare Vast VM lacks <code>rsync</code> , and GitHub clone would require a public repo or a token in the repo. <i>Abandoned: rsync; GitHub.</i>
Analysis local only	Saves metered GPU time, keeps analysis reproducible off-GPU, avoids pandas/matplotlib on the VM.
GSM8K id openai/gsm8k	Newer huggingface_hub rejects the bare canonical name.
Qwen3 dropped from the pilot	vLLM 0.6.6 predates the Qwen3 architecture and cannot load it; the pilot/full study run on the Llama pair(s). (The Qwen pair still appears in PRIMARY_PAIRS in code and must be removed before any Qwen run.)

6 Phase III - Pilot Studies

With the system built and validated GPU-free, we booked the RTX 3090 and ran a small pilot. Its purpose was a *mechanism check*-confirm the patch wires into the live worker, the controller is found in-process, DVFS actually toggles the SM clock, and the per-run JSON is written-*not* a results run. The first attempt exposed a subtle but fatal design flaw, which forced a redesign.

6.1 The first pilot and the constant- α failure

Pilot configuration. The first pilot was 7 runs on the LLaMA-3.1-8B / LLaMA-3.2-1B pair at a single draft length ($\gamma = 5$) on GSM8K: one vanilla/off reference plus all six DVFS conditions (off, adaptive_alpha, adaptive_entropy, two_level, coarse, fixed_low), 64 prompts, 256 max new tokens, one repetition. It ran successfully; all 7 result JSONs were produced. Table 6 shows the outcome.

Outcome and limitations. The plumbing worked: clock toggling was confirmed (draft 735 / verify 1935 MHz), all conditions produced ~769 phase transitions and ~13,132 tokens. But the pilot *could not validate the adaptive logic*, for a determinism reason. The mean acceptance rate came out at exactly $\alpha = 0.9208$ in *every* speculative condition. This is a direct consequence of temperature-0 + fixed-seed determinism (Section 5.6): the accept/reject sequence is bit-identical regardless of the DVFS condition, because DVFS changes clock timing, not computation. With α pinned at 0.9208 $\gg \alpha_{\text{high}} = 0.7$, the frequency mapper (Eq. 3) always returns f_{high} , so every adaptive policy permanently selects the high verify clock and collapses onto `two_level`. There was nothing left to differentiate the policies-they ran open-loop at a single operating point. The original 6-condition design could not exercise the thesis *even with perfect code*.

A second, headline observation. Even as a mechanism test, the pilot already surfaced the tension that would dominate the whole project: per-phase DVFS saved 14–16% GPU energy but cost +30% to +58% wall time, so *total* energy was flat-to-worse (+1% to +13%) and EDP was worse for every DVFS condition-net-negative at this tuning. Two comparisons within the table are diagnostic. `coarse` versus `adaptive_alpha` (both 769 transitions, differing only in the draft clock level) isolates the slowdown to the *low draft clock*, not switching overhead. And `fixed_low` being slowest (+58%) appeared,

Table 6: First pilot (7 runs, GSM8K, $\gamma = 5$, $n=64$ prompts). Percentages are versus the `spec_g5/off` baseline. Per-phase DVFS saved 14–16% GPU energy but cost +30% to +58% wall time, so *total* system energy was flat-to-worse. α was identical across every speculative condition.

Condition	Wall (s)	GPU kWh	Total kWh
vanilla/off	42.0	0.003977	0.006685
spec_g5/off (baseline)	31.3	0.002486	0.004585
spec_g5/coarse	36.7 (+17%)	0.002480 (−0%)	0.004790 (+4%)
spec_g5/adaptive_alpha	40.8 (+30%)	0.002077 (−16%)	0.004646 (+1%)
spec_g5/adaptive_entropy	40.6 (+30%)	0.002092 (−16%)	0.004632 (+1%)
spec_g5/two_level	41.1 (+31%)	0.002109 (−15%)	0.004736 (+3%)
spec_g5/fixed_low	49.5 (+58%)	0.002140 (−14%)	0.005203 (+13%)

at the time, to confirm that verify is genuinely clock-sensitive. (Initially, this suggested a compute-bound verify phase. However, later roofline analysis clarified that while verify is highly clock-sensitive in practice, it strictly operates in a memory-bound regime, Section 8.)

A retracted roofline over-claim. The same session attempted a first roofline capture under Nsight Compute (ncu), which *crashed* mid-capture on `fused_add_rms_norm_kernel`. It recorded only 8 distinct kernel launches-the layer-1 attention block only, with *no* MLP (gate/up/down_proj) kernels, dying before layer 2 of a 16-layer model. A naive sum of that partial capture gave $I \approx 0.082$ FLOPs/byte, which we briefly took as confirming draft memory-boundedness. This was *retracted*: an incomplete capture missing the highest-FLOP kernels biases intensity strongly downward, and the FP32 SASS counters used do not even count tensor-core GEMM FLOPs. We record this because it set a standing rule for the project-no memory-/compute-bound claim without one complete, non-crashing capture-and because it explains why the roofline had to be redone from scratch (Section 8). The ncu crash itself was later root-caused (Section A).

6.2 Pilot redesign: generating α variance

Methodological correction. The fix follows directly from the failure analysis. α cannot be moved by the DVFS condition (it cannot, by construction), but it *can* be moved by the two things that change the accept/reject sequence itself: the draft length γ (longer drafts propose tokens further out where draft and target diverge,

so α falls with γ), and the dataset (code and summarisation diverge from the draft more than formulaic math). By sweeping both γ and the evaluation datasets, we force α out of the saturated-high regime and down into the responsive transition band ($\alpha_{\text{low}} = 0.3 \dots \alpha_{\text{high}} = 0.7$). Within this active window, the adaptive policies evaluate to distinct clock frequencies, finally allowing them to diverge behaviorally from the static `two_level` baseline.

Revised experimental design. To address these limitations, we introduced three key modifications:

- (1) **Swept** $\gamma \in \{5, 12, 18\}$ across all three datasets (GSM8K / HumanEval / CNN-DailyMail), to spread α .
- (2) **Added a second draft model:** the same 8B-Instruct target paired with the 3.2-1B *base* (non-instruct) draft. Being less aligned with the instruct target, it accepts fewer tokens (lower α), widening the range tested and giving a genuine second data point rather than a repeated measurement. Both pairs share the same target, so the *vanilla* (no-SD) reference runs once per dataset and is shared across pairs.
- (3) **Trimmed to four active conditions** for the pilot (`off`, `two_level`, `adaptive_alpha`, `adaptive_entropy`); `fixed_low` and `coarse` were deferred to a full run as pure granularity ablations.

The pilot grew from 6 runs to a matrix of 75 measured runs plus roofline and the clock sweep. This redesigned matrix is the “main pilot” analysed next.

7 Phase IV - The Main Pilot (75 Runs)

7.1 Design

How the idea arose / why. The main pilot is a *broad sweep*. Rather than trying to explain the mechanism up front, we wanted a wide enough set of conditions to first pin down the basic facts: does per-phase DVFS save energy at all, and if so, how much does that cost in latency? The roofline check, the SM sweep, and the batch sweep all came later, and each was motivated by a specific question this pilot’s results raised.

How we implemented it. The matrix is

$$\underbrace{2}_{\text{draft pairs}} \times \underbrace{3}_{\gamma \in \{5, 12, 18\}} \times \underbrace{3}_{\text{datasets}} \times \underbrace{4}_{\text{DVFS policies}} = 72,$$

plus 3 shared *vanilla* runs (one per dataset, γ -independent) = 75 runs. The two pairs share the same 8B-Instruct target and differ only in the draft (1B-Instruct vs 1B-base). The three datasets are GSM8K, HumanEval, CNN-DailyMail. The four policies are `off`, `two_level`, `adaptive_alpha`, `adaptive_entropy`. All runs used the default engine settings (`max_num_seqs=8`, `max_model_len=2048`, 82 SMs), 64 prompts, 256 max new tokens, one repetition.

7.2 Results

Whether it worked. Per-phase DVFS saved GPU energy in every cell tested, across all three active conditions. Average GPU-energy savings were ~6–9%, with a mean latency overhead of 74.0%. The single best result was 9.6% GPU-energy savings (llama_8b_1b_base / spec_g18 / adaptive_entropy / GSM8K), which still carried a

Table 7: Main-pilot headline results (75 runs). Savings are GPU energy versus the per-cell off baseline. The base draft consistently out-saves the instruct draft; the three DVFS policies are statistically indistinguishable.

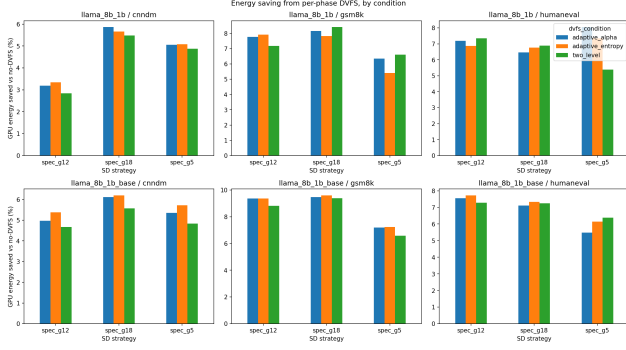
Quantity	Value
Mean GPU-energy saving (DVFS vs off)	~6–9%
Mean latency overhead	74.0%
Best single result (GPU energy saved)	9.6%
at latency overhead configuration	56.8%
	base / spec_g18 / adaptive_entropy / GSM8K
Spread of DVFS-policy differences	1–2 pp in most cells
adaptive_entropy vs two_level (max gap)	≤ 1.6 pp anywhere
GSM8K @ $\gamma=18$: base pair savings	9.4–9.6%
GSM8K @ $\gamma=18$: instruct pair savings	7.8–8.4%

56.8% latency penalty. Table 7 summarises the headline numbers; Figures 1–4 give the visual evidence.

Three findings from the pilot.

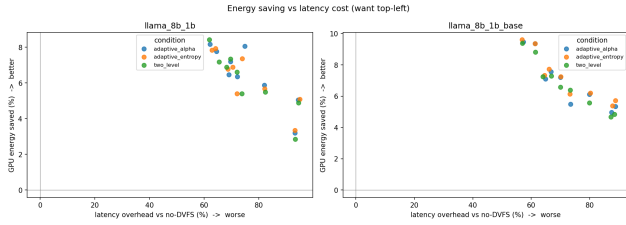
- (1) **The base draft out-saves the instruct draft** at matching γ /dataset cells (e.g. GSM8K @ $\gamma=18$: 9.4–9.6% vs 7.8–8.4%). The interpretation is that the base draft is less aligned with the instruct target, so it runs at a lower acceptance rate, pushing it further into the “responsive band” where the adaptive policies have room to actually adapt.
- (2) **The three DVFS policies barely separate.** `adaptive_entropy`, `adaptive_alpha`, and `two_level` track within ~1–2 percentage points of each other in almost every cell. The entropy-based leading signal never beats the trivial static policy by more than 1.6 percentage points anywhere. The added prediction complexity is not substantially better (discussed further in Section 11.1).
- (3) **The generality (“savings grow with γ ”) claim is only weakly supported.** The premise was that the benefit should grow with γ (a longer draft is a larger memory-bound fraction to downclock). The instruct pair is roughly flat with a slight rise at $\gamma=18$; the base pair shows a cleaner upward slope from ~6% ($\gamma=5$) to ~7.5% ($\gamma=18$). With only three γ points and percentage-point-scale noise at one repetition, this is suggestive but not established.

A reframing hiding in the absolute numbers. On absolute efficiency (Wh per 1000 tokens), the SD bars sit far below the *vanilla* bar-e.g. GSM8K vanilla ≈ 0.32 Wh/1k vs SD ≈ 0.17 Wh/1k, roughly halving energy per token-while the DVFS-vs-off gap within each SD cluster is a small further sliver. The big energy win in this domain is speculative decoding itself, per-phase DVFS is a real. We return to this in Section 12.



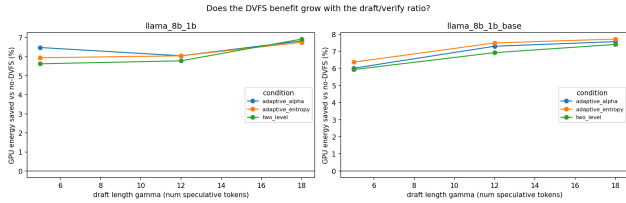
GPU energy saved (%) versus off, per DVFS condition, faceted by dataset. The three DVFS bars sit close together in every facet.

Figure 1: GPU energy saved (%) versus off, per DVFS condition, faceted by dataset. The three DVFS bars sit close together in every facet.



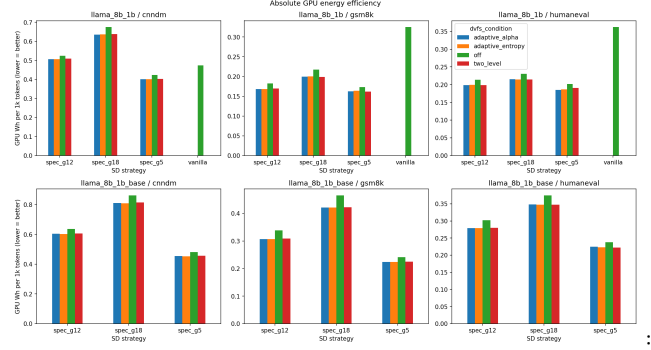
scatter of latency overhead (% , x, worse to the right) versus GPU energy saved (% , y, better upward). Points trend together, energy saved and latency cost scale together.

Figure 2: The cost side: every bit of energy saved comes with a proportional latency penalty.



energy saved (%) versus draft length γ , one line per DVFS policy. Lines for two_level, adaptive_alpha, and adaptive_entropy sit close together throughout; the base pair shows a cleaner upward slope than the instruct pair.

Figure 3: Does the benefit grow with the draft/verify ratio? Only weak, pair-dependent support.



absolute GPU Wh per 1k tokens. The vanilla bar towers over the SD bars (SD roughly halves energy per token); DVFS shaves a small further amount below off.

Figure 4: Absolute efficiency: the dominant win is SD itself, not DVFS.

8 Phase V - The Roofline Premise Check

The pilot established that savings exist but cost too much latency. The obvious next question is *why*, and the first suspect is the premise itself (Section 3). This phase tested it directly.

8.1 Profiling methodology

Roofline profiling runs *separately* from DVFS, at a fixed clock, so intensity is not confounded by clock changes. To profile the phases, a custom driver injects NVTX ranges at the exact draft and verify execution hooks utilized by the DVFS patch, ensuring perfect temporal alignment. During a minimal, deterministic generation task, `ncu` extracts per-range FLOPs and DRAM bytes into separate CSV files. Memory traffic is quantified using the `dram_bytes.sum` metric, while floating-point operations are calculated from the FP32 SASS instruction counters (`fadd + fmul + 2xfma`). The analysis computes I = FLOPs/bytes per phase and classifies each against the ridge point I^* (Eq. 2). For the RTX 3090 we use peak ≈ 71 TFLOP/s and 936 GB/s, giving $I^* \approx 75.9$ FLOPs/byte.

8.2 Baseline results: The theoretical premise fails on unmodified hardware

On the unmodified RTX 3090 (all 82 SMs), the measured intensities are:

Phase	I (FLOPs/byte)	Regime
ridge point I^*	75.85	-
draft	41.09	memory-bound ($I < I^*$)
verify	61.25	memory-bound ($I < I^*$)

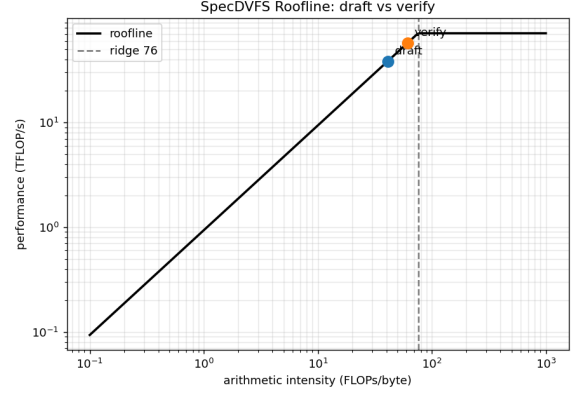
Both phases are below the ridge-both memory-bound. This directly contradicts the premise, which required $I_{\text{draft}} < I^* \leq I_{\text{verify}}$. Verify gets close ($61.25/75.85 = 81\%$ of the way to the ridge) but does not cross it. According to the hypothesis, verify should have been compute-bound ($I \geq 75.9$); it is not. Figure 5 shows the baseline roofline.

8.3 Root cause analysis and methodological robustness

The premise fails because verify does not reach the ridge. Two honest caveats apply, and neither rescues the premise. First, this capture was taken at a small draft length and minimal decode width; verify’s intensity should *rise* with more concurrent sequences (more FLOPs amortising the same weight reads), so a wider measurement could push verify higher-this is exactly the motivation for the batch-size experiment (Section 10). Second, the FP32 counters undercount `bf16/tensor-core` math, which would bias the *absolute* FLOP figures; but this bias applies to both phases and does not change the qualitative ordering that both sit left of the ridge. The verdict-premise false on stock hardware-holds.

9 Phase VI - The SM-Restriction Experiment

This experiment is designed as a direct mechanism test, isolating whether the compute/memory split itself drives the observed savings.



baseline roofline on the unmodified RTX 3090. The ridge is at 75.85 FLOPs/byte; both draft (41.09) and verify (61.25) sit to its *left*-both memory-bound. This is the core negative result: the theoretical justification for per-phase DVFS does not hold on stock hardware.

Figure 5: Baseline roofline (82 SMs): both phases are memory-bound.

9.1 How the idea arose

The roofline showed verify just below the ridge on stock hardware. In a discussion of this result, Keshav suggested a way to push verify across the ridge without changing the workload: reduce the number of GPU cores/threads available to the process. The insight rests on Eq. 2: because $I^* = \text{peak FLOP/s} / \text{peak bandwidth}$ is a property of the *machine*, restricting the number of active streaming multiprocessors (SMs) cuts peak FLOP/s roughly linearly while leaving DRAM bandwidth essentially unchanged. So I^* falls and the ridge slides *down* through the two fixed, measured phase intensities, while the arithmetic intensity of the phases themselves does not change (same kernels, same FLOPs, same bytes). Thus the SM restriction gives the window for testing the premise.

9.2 Why we expected it to work

Using the pilot’s own measured intensities on the RTX 3090 (71.0 TFLOP/s, 936 GB/s), the ridge as a function of active SM count N (out of 82) is

$$I^*(N) = \frac{71.0 \times (N/82)}{0.936} \quad [\text{FLOPs/byte}]. \quad (5)$$

The premise $I_{\text{draft}} < I^*(N) \leq I_{\text{verify}}$ becomes $41.1 < I^*(N) \leq 61.3$, which holds for $N \in [45, 63]$, i.e. 54–77% of the GPU. This window is *narrow and bounded on both sides*: below ~ 45 SMs the ridge falls past *draft*’s intensity too, so both phases become compute-bound and the premise breaks the *other* way. We therefore chose an SM sweep that *brackets* the window rather than assuming it: {82, 64, 56, 48, 40}. If the compute/memory split is the mechanism, savings should be small/negative outside the window (82, and below ~ 45) and *peak* inside it (56, 48).

9.3 How we implemented SM restriction

Mechanism: NVIDIA MPS active-thread percentage. We cap SMs with the environment variable `CUDA_MPS_ACTIVE_THREAD_PERCENTAGE`, which limits the fraction of SMs a client may use. It requires the MPS control daemon to be running and is read *once*, when the client process creates its CUDA context. Two consequences drove the design:

- The variable must be set *before* torch/vLLM initialise CUDA, so it is set at process start and it cannot be changed mid-process. So an SM sweep must run *one subprocess per SM level*. `run_sm_sweep()` spawns one child per level automatically.
- **Without the MPS daemon the variable is silently ignored.** All five “levels” would then run on the full GPU and produce a plausible-looking *fake* sweep. A `preflight()` guard therefore aborts if the daemon is down and a server-readiness check flags it earlier still. We verified the env var actually propagates through the subprocess boundary (each child stamps its own level into its result JSON).

Alternatives considered and rejected. MIG partitions a data-center Ampere GPU into instances with fractional SMs and proportionally reduced bandwidth, but the consumer RTX 3090 (GA102) *does not support MIG*. CUDA Green Contexts (`cuGreenCtxCreate`, CUDA 12.4+) give exact SM-count partitions rather than a percentage but the green context must be made current before vLLM builds its own context inside the worker. This would require in-process CUDA-API change on vLLM’s worker init. MPS achieves the same experimental effect with no such change, so it is preferred. A third option, rewriting kernel grid dimensions via an `LD_PRELOAD` shim, is far too fragile (it breaks whenever kernel configs change) and was thus not preferred.

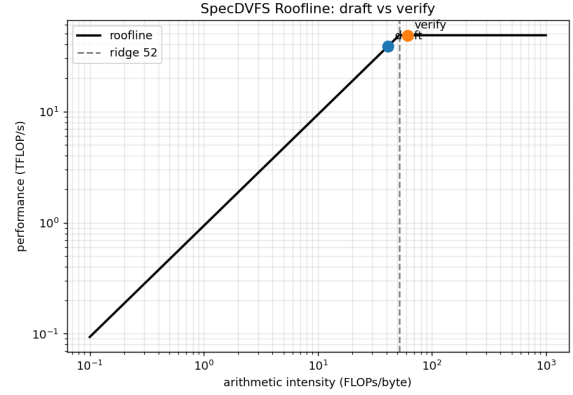
Why only one roofline capture was needed. Arithmetic intensity depends only on the computation. It is not dependent on how many SMs run it. So a single unrestricted capture is enough: the ridge at each restricted N is computed analytically from that one measurement.

Configuration. The SM sweep is a single fixed cell: pair `llama_8b_1b`, strategy `spec_g18` ($\gamma=18$, not swept here), GSM8K only, varied across 5 SM levels $\{82, 64, 56, 48, 40\} \times 4$ DVFS conditions = 20 runs, at the default batch/context (`max_num_seqs = 8`, `max_model_len = 2048`) matching the main pilot. This is not a deployment scenario as nobody usually restricts SMs in production. It is a controlled setting to ask “Would the predicted benefit show up if the premise were true?”

9.4 Whether it worked - forcing the premise true was not enough

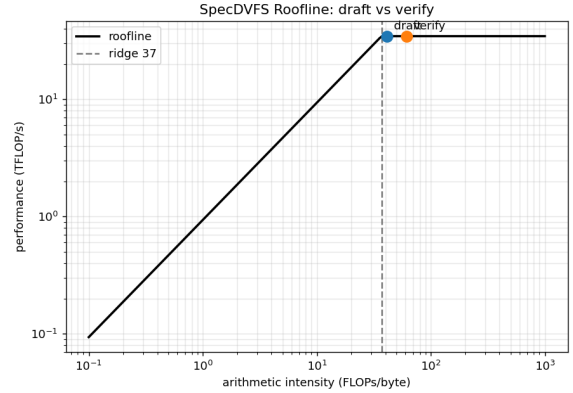
The ridge slides down through the phase intensities as SMs are removed, crossing verify between 64 and 56 SMs and crossing draft below 45 (Figures 6–7). The premise is successfully switched on at 56/48 SMs and broken the other way at 40. Table 8 gives the full premise table and per-condition savings.

The problem for the hypothesis: If the compute/memory split were the mechanism driving DVFS savings, the savings curve should be *humped*: near-zero at 82 (premise false), peaking inside 56/48



: with the GPU restricted to 56 of 82 SMs, the ridge drops to 51.80. Verify (61.25) now sits to its *right* (compute-bound) while draft (41.09) stays to its left—the premise made true under controlled conditions.

Figure 6: 56 SMs: verify crosses to compute-bound; the premise holds.



: at 40 SMs the ridge falls to 37.0 and *draft* (41.09) crosses too—both phases compute-bound. The premise window has a far edge; the boundary breaks in both directions exactly as the ridge math predicts.

Figure 7: 40 SMs: draft crosses too; the premise breaks the other way.

(premise true), falling at 40 (premise false again). Instead it is *monotonically decreasing*, from +6.70% at 82 SMs down to −1.84% at 40 SMs, crossing zero right in the middle of the window where the premise holds. Figure 8: the savings curve declines straight through the shaded “premise-true” window rather than peaking inside it.

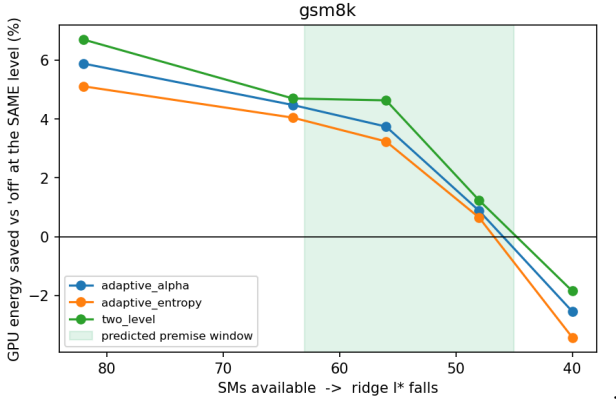
9.5 What is actually happening instead

The curve is explained by decomposing energy into power $\times$ time at each SM level (Table 9). Power reduction stays roughly flat across the sweep (33% $\rightarrow$ 40%, a +7 pp change), while the wall-time penalty

Table 8: SM-restriction experiment: premise status and GPU-energy savings versus off at each SM level (GSM8K, spec_g18, batch= 8; measured $I_{\text{draft}} = 41.09$, $I_{\text{verify}} = 61.25$). The premise is successfully forced true at 56/48 SMs, yet savings decline monotonically instead of peaking there.

SMs	% of GPU	ridge I^*	draft	verify		premise	two_level	adaptive_alpha	adaptive_entropy
82	100%	75.85	mem	mem	NO-GO (both memory-bound)		+6.70%	+5.89%	+5.11%
64	78%	59.24	mem	mem	NO-GO (marginal, just outside)		+4.70%	+4.48%	+4.05%
56	68%	51.80	mem	compute		GO	+4.64%	+3.74%	+3.24%
48	59%	44.40	mem	compute		GO	+1.24%	+0.88%	+0.65%
40	49%	37.00	compute	compute	NO-GO (both compute-bound)		-1.84%	-2.54%	-3.42%

Approach 2: does DVFS help more inside the premise window?



GPU-energy saved (%) versus SM count, with the “premise-true” window shaded. The curve declines *monotonically* and crosses zero inside the shaded window—no hump, no response to the premise boundary at all.

Figure 8: The key contradiction: savings do not respond to the premise boundary.

Table 9: Energy decomposition across the SM sweep (two_level vs off). Power reduction is roughly flat; the wall-time penalty grows $\sim 5\times$ faster, so net energy savings decay and flip sign.

SMs	wall-time penalty	power reduction	net energy	off wall (s)	off tok/s
82	+54.7%	-33.3%	-6.70%	16.1	400.8
64	+65.5%	-36.7%	-4.70%	18.2	359.4
56	+68.4%	-37.5%	-4.64%	18.7	355.4
48	+79.0%	-39.7%	-1.24%	19.5	333.1
40	+87.7%	-40.3%	+1.84%	21.6	305.9

grows sharply (55% $\rightarrow$ 88%, a +33 pp change), roughly five times as fast in relative terms. As SMs are removed, the time cost outruns the power benefit at an accelerating rate, so net savings erode and eventually flip sign.

Why the time penalty grows as SMs shrink. The draft phase runs pinned at $f_{\text{low}} = 735$ MHz throughout the sweep. A genuinely memory-bound kernel should be waiting on DRAM and largely indifferent to the core clock. Removing SMs then takes compute away from a phase that is already starved for it, despite the roofline

Table 10: Predicted verify arithmetic intensity versus batch size at $\gamma=18$. The ridge (75.9) is predicted to be crossed between batch 8 and 11.

batch	4	8	11	16	22
predicted I_{verify}	29.3	58.5	80.5	117.1	161.0
vs ridge 75.9	mem	mem	compute	compute	compute

characterising it as memory-bound. So the penalty compounds rather than staying constant.

Determinism check. Across all 20 SM-sweep runs, total token counts are bit-identical across the four DVFS conditions at fixed settings and α varies by under 1%. This confirms that DVFS is changing timing, not computation. This makes the energy comparison meaningful.

EDP. On the combined energy-delay product, off wins at every SM level, from a +44% penalty at 82 SMs to a +91% penalty at 40 SMs. Restricting SMs never makes DVFS strategies explored worthwhile on the combined metric.

10 Phase VII - The Batch-Size Experiment and the Roofline Capture Failure

10.1 Motivation

SM restriction moves the ridge (the ceiling) while the measured point stays put; it is a controlled instrument, not a deployment scenario. Batch size is the *realistic* counterpart: it is a parameter that real serving systems actually vary, and raising it moves the *measured point* rightward on the roofline while the ridge stays fixed. When verify processes multiple sequences in one batched pass, the model’s weight matrices are read from VRAM once and reused across every sequence’s matrix multiplication, so bytes moved stay roughly fixed. Meanwhile the FLOPs scale with the number of sequences due to which arithmetic intensity rises.

Predicted crossing. At $\gamma=18$, verify scores $\gamma + 1 = 19$ tokens per sequence. Modelling $I_{\text{verify}} \approx 58.5 \times (\text{batch} \times 19) / 152$ predicts a crossing of the ridge (75.9) between batch 8 and 11 (Table 10)—two levels below the crossing, three above.

10.2 Implementation

The batch sweep is 40 runs: 5 batch levels {4, 8, 11, 16, 22} (set via `max_num_seqs`) $\times$ 2 datasets (GSM8K, HumanEval) $\times$ 4 DVFS conditions. Two design choices keep the sweep clean:

- **Context length fixed, not varied with batch.**

`max_model_len` is held at 1024 throughout (down from the

Table 11: Batch-size experiment: joint GPU-energy savings, latency overhead, and EDP ratio (vs off, lower EDP is better, 1.0 is breakeven) for two_level policy. off wins on EDP at every level; the best case is a 1.31× penalty.

batch	dataset	energy saved	latency	EDP ratio
4	GSM8K	+9.5%	+55.8%	1.41×
4	HumanEval	+10.3%	+56.6%	1.41×
8	GSM8K	+7.2%	+55.7%	1.44×
8	HumanEval	+6.4%	+59.1%	1.49×
11	GSM8K	+7.1%	+54.0%	1.43×
11	HumanEval	+7.2%	+55.5%	1.44×
16	GSM8K	+8.3%	+51.1%	1.39×
16	HumanEval	+7.3%	+53.2%	1.42×
22	GSM8K	+10.5%	+46.2%	1.31×
22	HumanEval	+8.3%	+50.3%	1.38×

pilot’s 2048) to leave enough KV-cache headroom for the higher concurrency levels. It is deliberately *not* tuned per batch level, because doing so would change how much of each CNN/DailyMail article gets truncated at each level, confounding the effect of *batch size* with the effect of *how much of the prompt the model sees*.

- **CNN/DailyMail excluded from this sweep.**

At `max_model_len` = 1024, the prompt budget after reserving 256 tokens for generation and a 48-token margin is $1024 - 256 - 48 = 720$ tokens. GSM8K prompts (~60–150 tokens) and HumanEval prompts (~100–300 tokens) fit comfortably within this budget untruncated; CNN/Daily-Mail articles (~500–1800 tokens) do not, so including them would silently vary how much of the source article each run actually sees.

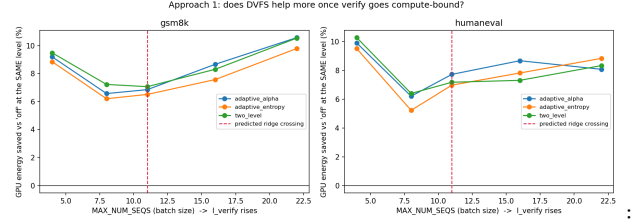
10.3 The energy result

The energy data is valid, and batch size genuinely varied. Throughput scales cleanly from 238 tok/s (batch= 4) to 591 tok/s (batch= 22), confirming real concurrency (the energy runs use 32 real prompts, so batching truly occurs). Table 11 gives the joint energy/latency/EDP result for `two_level`.

Findings. Energy savings show a mild U-shape across batch levels: highest at the extremes (~9–10.5% at batch 4 and 22), lowest in the middle (~5–7% around batch 8). Latency overhead is substantial throughout (46–61%), decreasing slightly as batch grows (higher throughput partially offsets the DVFS time penalty). On EDP, *off* wins at every batch level tested—best case for DVFS is a 1.31× EDP penalty (batch= 22), worst 1.52× (batch= 8, *adaptive_entropy*). This is consistent with every other part of the study: real energy savings exist, but the latency cost outweighs them on the combined metric regardless of batch size.

10.4 The roofline capture failure

Discovery. While cross-checking the batch-size *roofline* data against the energy sweep, we noticed that batch= 8 and batch= 11 produced almost *identical* verify arithmetic intensity: 61.24 versus 61.25 FLOPs/byte, a difference of 0.016%—well within noise. That was suspicious on its face. The whole point of the experiment was that raising batch size should *measurably* raise *verify*’s intensity, by amortizing the same weight reads over more FLOPs.



GPU-energy saved (%) versus batch size (4 to 22), two dataset panels, showing the mild U-shape (best at the extremes). This energy data is valid; what is still open is the mechanistic *explanation* of the U-shape (see the roofline bug below).

Figure 9: Batch-size energy sweep: mild U-shape, EDP never below 1.31×.

Root Cause. We traced the problem to the capture script: every roofline capture used a fixed `-n-prompts 1`, regardless of the `max_num_seqs` value—the batch-size setting actually under test. `max_num_seqs` is a scheduler *admission cap*: it only takes effect when enough concurrent prompts exist to admit against it. With exactly one prompt in flight, there was never more than one sequence for the cap to restrict, so batch= 8 and batch= 11 ran the identical single-sequence computation regardless of what the run was labelled. The roofline captures had never actually varied the batch size at all.

Compounding Impact. Investigating further, we found the same flaw in the `sm82` baseline capture—the single measurement from which *all four* SM-restriction roofline verdicts (Section 9) were derived. It, too, had been captured at batch= 1, while the real SM-sweep *energy* runs use batch= 8. That means the roofline verdicts in Table 8 rest on an intensity measured at the wrong batch size.

Scope of Impact. The batch-sweep *energy* data (Table 11) is unaffected: those runs used real prompts, and batching genuinely occurred (throughput scaled from 238 to 591 tok/s). What failed was specifically the *roofline arithmetic-intensity confirmation*, whose job was to explain *why* the energy U-shape looks the way it does—whether it correlates with *verify* crossing into compute-bound territory. We simply never got that mechanistic explanation. The result is best read as missing, not negative: we have no evidence against the compute-bound explanation, only a gap where the confirming measurement should be.

Resolution and Current Status. We fixed the root cause—matching `n_prompts` to the actual batch size under test—verified the fix against the exact case that exposed the bug, and expanded the captured batch levels from {8, 11} to {4, 8, 11, 22} to better match the energy sweep. The corrected re-capture, however, has not gone cleanly. Progress was slowed by unrelated cloud-GPU reliability problems: long stalls, a dropped SSH connection mid-run, and one instance stuck in an unexplained high-power state that even a reboot would not clear, forcing a switch to a fresh machine. More tellingly, once we did get a clean run through the fixed pipeline, the result came back unchanged—which points to a second, still unidentified bug rather than confirming the fix. We therefore treat the roofline confirmation for both the SM and batch experiments as an open item (Section 13). Our headline conclusions do not depend on it: the SM

experiment’s central contradiction—that savings do not respond to the premise—is an *energy* result, and the ridge-crossing it relies on describes a property of the computation that higher concurrency should, if anything, only reinforce.

Related Issue: The ncu Hang. The crashed first capture (Section 6.1) was eventually root-caused as well. Nsight Compute’s default kernel-replay mode rewinds and re-executes individual kernels in isolation to gather counters. vLLM’s speculative decoding, however, reads the rejection sampler’s acceptance count back to the CPU *mid-iteration* to decide control flow. That means the CPU ends up blocked on a device-to-host copy of a value produced by the very kernel ncu is mid-replay on: a circular wait that parks the process at 0% GPU utilisation indefinitely. The fix is `-replay-mode application`, which re-runs the whole tiny, deterministic program once per pass instead of rewinding individual kernels, so nothing can deadlock. The only cost is that the model reloads once per pass, adding seconds rather than causing an indefinite hang. Full details are in Section A.

11 Phase VIII - Diagnostics and Open Issues

11.1 Adaptive policy comparison: sophistication does not pay

To evaluate whether additional policy sophistication translates into greater energy savings, we compared two adaptive DVFS policies against the simple static policy (`two_level`). Across all conditions evaluated, however, the three policies were statistically indistinguishable. The entropy-based policy (`adaptive_entropy`) never outperformed the static policy by more than approximately 1.6 percentage points in any pilot configuration, and the same pattern persisted across both the SM-restriction and batch-size sweeps. This result is consistent with the roofline analysis: when the verify phase remains memory-bound, there is little opportunity for an adaptive verify-frequency policy to exploit. Consequently, the additional implementation complexity of the entropy-based approach—including an `lm_head` forward hook, per-model-pair calibration, and a fitted prediction curve—does not provide measurable benefits over a simple static threshold rule.

A narrative caveat worth stating. `adaptive_entropy` has a sound theoretical grounding (GELATO’s entropy $\rightarrow \alpha$ relationship, Section 5.5), and the calibration machinery works. The negative result is not that the entropy signal is wrong; it is that, in a regime where verify never becomes compute-bound, there is no headroom for *any* verify-clock policy to exploit, sophisticated or not.

11.2 An unresolved measurement issue: CodeCarbon vs NVML

The issue. Recall that every run records GPU energy two ways (Section 5.6): CodeCarbon (software estimation) and NVML (100 Hz power sampling $\times$ wall time). On the SM sweep, *the two methods disagree on the sign of net savings at every SM level* (Table 12): CodeCarbon reports `two_level` saving 1–7%, while NVML reports it costing 3–12%. The disagreement (~ 10 percentage points) is larger than the effect itself.

Characterizing the measurement discrepancy. Notably, this divergence is isolated entirely to the baseline `off` condition; the

Table 12: CodeCarbon vs NVML net-savings estimates for `two_level` across the SM sweep. The tools disagree on the sign at every level, but agree on the direction: savings decline as SMs drop under either tool.

SMs	CodeCarbon	NVML (power $\times$ time)
82	+6.7% (saves)	−3.1% (costs)
⋮		
40	(declining) −1.8% (costs)	(declining) −12.1% (costs)

active DVFS conditions show no such disagreement. Specifically, the energy ratio between the two reporting methods is ~ 1.08 – 1.10 for the `off` runs, whereas it converges tightly to ~ 0.98 – 0.99 during the DVFS runs. We propose two plausible hypotheses to explain this baseline-specific phenomenon, though neither has been definitively confirmed:

- NVML’s 100 Hz sampler may undersample real power spikes during `off` runs specifically. `off` shows far more distinct clock levels (10–17, from the GPU’s own unlocked boost algorithm reacting to temperature/load) than DVFS runs; if power correlates with those rapid clock changes, NVML could be missing peaks and underestimating `off`’s true energy.
- Alternatively, CodeCarbon could be systematically inflating estimates for short, bursty, high-power runs. This cannot yet be ruled out.

12 Discussion - What the Evidence Points To

Three independent probes: a broad pilot, an artificially engineered “best case,” and a more realistic batch-size sweep—examined the per-phase DVFS hypothesis from different angles. They converge on the same answer.

12.1 The savings are real but the trade is unfavourable

While per-phase DVFS yields consistent GPU-energy savings of 3–10% across 95+ runs, these reductions are entirely eclipsed by a severe latency overhead of 46–88%. On the energy–delay product (EDP), the no-DVFS baseline wins at every operating point we tested (SM level, batch level): the best case for DVFS is a $1.31\times$ EDP penalty. Ultimately, any energy it saves is not free; it is bought with far more latency than it is worth on the combined metric.

12.2 The premise is false on unmodified hardware and even forcing it true did not help

On unmodified hardware, the theoretical premise—draft memory-bound, verify compute-bound—does not hold, both phases measure memory-bound (draft 41.1, verify 61.3, ridge 75.9). To test the premise further, we used SM restriction to *force* the verify phase to become compute-bound. Even under these ideal theoretical conditions, the predicted benefit did not appear. Instead, energy savings declined monotonically rather than peaking in the premise-true window.

The roofline framing therefore cannot be used to explain the observed savings.

12.3 A plausible explanation

Our current findings suggest that the observed energy savings may be explained by a conventional power–latency trade-off associated with downclocking a kernel that is *clock-limited in practice*, rather than by a transition between compute- and memory-bound execution. Decomposing energy into power×time across the SM sweep (Section 9.5) shows that power reduction remains approximately constant while the latency penalty grows nearly fivefold faster, causing the initial energy savings to diminish and eventually reverse as compute resources are restricted. This observation points towards a power–latency trade-off as a plausible explanation for the measured behaviour. However, this hypothesis has not yet been investigated in sufficient depth, and further microarchitectural analysis will be required to establish the underlying mechanism conclusively.

12.4 Sophistication did not pay here

The three adaptive policies (static two_level, lagging adaptive_alpha, leading adaptive_entropy) remain indistinguishable across every condition tested; the added prediction complexity does not beat a trivial threshold. This is a direct consequence of the premise failing: without a genuinely compute-bound verify phase, there is no computational headroom for an adaptive clocking policy to exploit.

13 Threats to Validity and Limitations

We state the limitations plainly; several are already reflected in the caveats above.

- **No error bars due to Single repetitions of runs** The pilot, SM and batch data are single-rep. The SM-sweep *trend* is credible because it is monotonic across five independent points. However, individual point-to-point differences cannot be statistically defended.
- **The roofline confirmation for the batch experiment is an open item.** The capture bug (Section 10.4) was fixed, but a clean corrected re-capture has not completed as there is still suspected remaining bug. Our energy-based conclusions do not depend on it.
- **CodeCarbon vs NVML disagree on the sign of net savings** (Section 11.2). The direction is robust under both tools, but the absolute sign at full GPU is uncertain pending a re-analysis of existing data.
- **f_{low} was not optimal, and was not swept in the full matrix.** A better f_{low} would reduce the latency penalty (and the energy savings); we have not yet re-run the matrix at the improved value, so the combined-metric verdict at optimal f_{low} is not directly measured.
- **Hardware and engine specificity.** Results are for one RTX 3090 under vLLM 0.6.6 with a pinned stack, at $max_model_len \leq 2048$, $max_num_seqs \leq 22$, $enforce_eager=True$. Other GPUs (different ridge points), engines, longer contexts, and CUDA-graph execution could shift the quantitative picture.

- **Model-pair coverage.** All runs done on two draft/target pairs, both Llama-family sharing one 8B-Instruct target. A weaker draft (e.g. a $\sim 190\times$ ratio Vicuna-13B/68M pair) would give lower, more variable α and wider coverage. However this needs more VRAM than the 24 GB card allows.
- **SM restriction is not a deployment scenario.** It is a controlled setting by construction. The batch-size experiment is more deployment-realistic, however it’s roofline confirmation is the open item above.

14 Conclusion and Future Work

14.1 Conclusion

SpecDVFS asked whether per-phase DVFS can make LLM speculative decoding more energy-efficient at negligible latency cost, as the standard memory-bound-draft / compute-bound-verify intuition predicts. We built a complete control system and measured 95+ configurations on a data-center-class GPU. The answer based on these pilot runs seems negative. The premise is false on stock hardware (both phases memory-bound); per-phase DVFS does save real GPU energy (3–10%) but at a latency cost (46–88%) that makes the no-DVFS baseline win on EDP everywhere we looked; forcing the premise true via SM restriction does not produce the predicted benefit (savings decline monotonically through the premise window); and a sophisticated entropy-based policy never beats a trivial static one. The effect we measured is an ordinary power/latency trade of downclocking a clock-limited-in-practice draft phase, not the compute/memory effect we hypothesised.

14.2 Remaining Work and Future Directions

Several aspects of this work remain unfinished due to the project’s scope and time constraints. We list them below in roughly the order of priority.

- (1) **Complete the batch-size roofline capture.** The roofline confirmation for the batch-size experiment (Section 10.4) was never obtained: the capture bug was root-caused and fixed, but the corrected re-capture did not complete, blocked by cloud-GPU reliability issues and a suspected remaining bug we have not yet located. This is the single most important re-run outstanding, since it is what would mechanistically explain the batch-sweep energy U-shape (Section 10.3) rather than only describing it.
- (2) **Sweep f_{low} inside the full matrix - but we do not expect a large effect.** Re-running the main pilot and the SM/batch sweeps at a higher f_{low} (e.g. ≥ 990 MHz) is a mechanically straightforward next step. We flag, however, that because energy and latency move *together* across the f_{low} sweep, we do not expect this to change the qualitative EDP verdict of Section 12-only to shift the absolute numbers on both sides of the trade.
- (3) **Broaden model-pair and quantization coverage.** All results in this report are bf16 on two Llama-family pairs sharing one target (Section 13). A weaker draft/target pair (lower, more variable α) and a working quantized configuration (higher arithmetic intensity, and so nominally more favourable to the compute-bound premise) would both test

whether our conclusions are an artifact of the specific pairs and precision we had access to, or hold more generally.

- (4) **Extensively investigate why the hypothesis failed, not just that it did.** This report establishes the negative result and traces it to a power/latency trade of a clock-limited-in-practice draft phase (Section 9.5), but that mechanistic account was assembled from diagnostics run alongside the main campaigns rather than from an investigation designed around the question. A dedicated follow-up-finer-grained per-kernel clock/latency profiling of the draft phase, and a closer look at whether the memory-bound roofline classification itself is being measured correctly at this scale-would let us say with more confidence *why* the compute/memory model breaks down here, rather than only that it does.
- (5) **Reconsider the intervention point: iterate on existing DVFS mechanisms instead of a custom monkey-patched controller.** This study built and measured a bespoke per-phase controller wrapped around vLLM’s speculative-decoding worker (Sections 5.2–5.4). An alternative direction for future work is to set that approach aside and instead examine the DVFS behaviour already present in the stack—the GPU’s own driver-level clock/boost management and any power-aware scheduling already exposed by the serving engine—and iterate on *those* mechanisms directly. Given that our results show the latency penalty, not the availability of energy savings, is the binding constraint, tuning within an existing, already-integrated control loop may be a more promising route to minimising latency overhead than adding a new external controller on top.

A The ncu Hang (Diagnosed)

The first roofline capture (Section 6.1) hung at 0% GPU and had to be aborted; a naive interpretation of its partial output was retracted. The root cause is an interaction between Nsight Compute’s replay mechanism and speculative decoding’s control flow. ncu’s default kernel replay rewinds and re-executes individual kernels in isolation to gather counters. But vLLM speculative decoding reads the rejection sampler’s acceptance count back to the CPU *mid-iteration* to choose control flow, so the CPU blocks on a device→host copy of a value produced by the very kernel ncu is mid-replay on. That circular wait deadlocks the process. The fix is `-replay-mode application:` instead of rewinding individual kernels, ncu re-runs the whole tiny, deterministic (temperature-0, seed-42) program once per pass, so nothing is rewound and it cannot deadlock. The only cost is that the model reloads each pass-seconds, not an indefinite hang-and a one-time driver warmup before the profiled region keeps the capture in steady-state decode. If the GPU’s performance counters are admin-locked (ERR_NVGPUCTRPERM, common on cloud VMs), the same command is re-run under `sudo env "PATH=$PATH" ncu`

...

B Reproducibility Summary

Key constants (experiments/run_experiment.py): $f_{\text{high}} = 1935$ MHz, $f_{\text{low}} = 735$ MHz, $f_{\text{floor}} = \lfloor 0.6 \cdot 1935 \rfloor = 1161$ MHz (though `draft_freq()` returns f_{low} directly), seed = 42, max new tokens = 256, warm-up prompts = 5, cool-down = 30 s, monitor = 100 Hz, calibration $\gamma = 5$.

Engine: vLLM 0.6.6, transformers < 5.0.0, enforce_eager=True, max_model_len = 2048 (pilot) / 1024 (batch sweep), max_num_seqs = 8 (default). Models: target meta-llama/Llama-3.1-8B-Instruct; drafts meta-llama/Llama-3.2-1B-Instruct (instruct pair) and meta-llama/Llama-3.2-1B (base pair). Datasets: GSM8K (openai/gsm8k), HumanEval, CNN/DailyMail.

References

- [1] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast Inference from Transformers via Speculative Decoding. In *Proc. ICML*, 2023.
- [2] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating Large Language Model Decoding with Speculative Sampling. *arXiv preprint arXiv:2302.01318*, 2023.
- [3] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proc. SOSP*, 2023.
- [4] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An Insightful Visual Performance Model for Multicore Architectures. *Communications of the ACM*, 52(4):65–76, 2009.
- [5] Rohit Dutta, Paramita Koley, Soham Poddar, Janardan Misra, Sanjay Podder, Naveen Balani, Saptarshi Ghosh, and Niloy Ganguly. Benchmarking the Energy Savings with Speculative Decoding Strategies. In *Findings of the Association for Computational Linguistics: EACL 2026*, pages 4737–4748, Rabat, Morocco, 2026. Association for Computational Linguistics.
- [6] W. Yang and S. Xia. PELM: Power Efficient On-Device LLM Inference with Speculative Decoding and Dynamic Voltage Frequency Scaling. In *Proceedings of the 2026 ACM/IEEE International Conference on Embedded Artificial Intelligence and Sensing Systems (EAISS)*, pages 438–451, 2026. doi:10.1145/3774906.3802783.
- [7] Zengzipeng Tang, Yuxuan Sun, Wei Chen, Jianwen Ding, and Bo Ai. GELATO: Generative Entropy- and Lyapunov-based Adaptive Token Offloading for Device-Edge Speculative LLM Inference. *arXiv preprint arXiv:2605.10124*, 2026.
- [8] Paul Joe Maliakel, Shashikant Ilager, and Ivona Brandic. Characterizing LLM Inference Energy-Performance Tradeoffs across Workloads and GPU Scaling. *arXiv preprint arXiv:2501.08219*, 2025.
- [9] Andreas Kosmas Kakolyris, Dimosthenis Masouros, Sotirios Xydis, and Dimitrios Soudris. SLO-Aware GPU DVFS for Energy-Efficient LLM Inference Serving. *IEEE Computer Architecture Letters*, 23(2):150–153, 2024.
- [10] Qunyou Liu, Darong Huang, Marina Zapater, and David Atienza. Green-LLM: SLO-Aware Dynamic Frequency Scaling for Energy-Efficient LLM Serving. *arXiv preprint arXiv:2508.16449*, 2025.